BookNest

Smart Book Lending & Reading Community

Presented by: [Asmit Nayan] | [RA2411030030065]

[Kanhaiya Kumar] | [RA2411030030068]

[Sarthak Srivastava] | [RA2411030030047]

The Problem

Manual & Fragmented

Book lending is tracked on paper or spreadsheets — error-prone, slow, and hard to scale.

No Single Source of Truth

Users, books, lendings, and reviews exist in silos with no unified system to manage them.

Zero Visibility

Librarians and members have no real-time view of availability, history, or due dates.

Libraries without a centralized DBMS lose an average of **15–20%** of records to human error annually — making digital management not optional, but essential.

Introducing BookNest

A full-stack, database-driven platform that centralizes every aspect of book lending and community engagement.

Book Catalog

Add, search, and manage the entire book inventory in one place.

Member Management

Register and track members, borrowing history, and account status.

Lending Engine

Issue and return books with automatic due-date tracking.

Reviews & Ratings

Community-driven reviews to help readers discover great books.

Key Features

Add & Search Books

Instantly add new titles or search the catalog by title, author, or genre.

Member Registration

Onboard new members with full profile and borrowing eligibility tracking.

Issue & Return

One-click lending and return workflows with date logging built in.

Tech Stack

BookNest is built on a modern, production-grade technology stack designed for reliability and extensibility.

Frontend

HTML / CSS / JavaScript

Backend

FastAPI (Python)

ORM

SQLAlchemy

Database

PostgreSQL

FastAPI's async capabilities and SQLAlchemy's ORM layer ensure clean separation of concerns — making the system both performant and maintainable.

System Architecture

All client-server communication occurs via RESTful API calls — keeping the frontend fully decoupled from business logic and data storage.

Presentation Layer

HTML/CSS/JS renders UI and sends HTTP requests to the API.

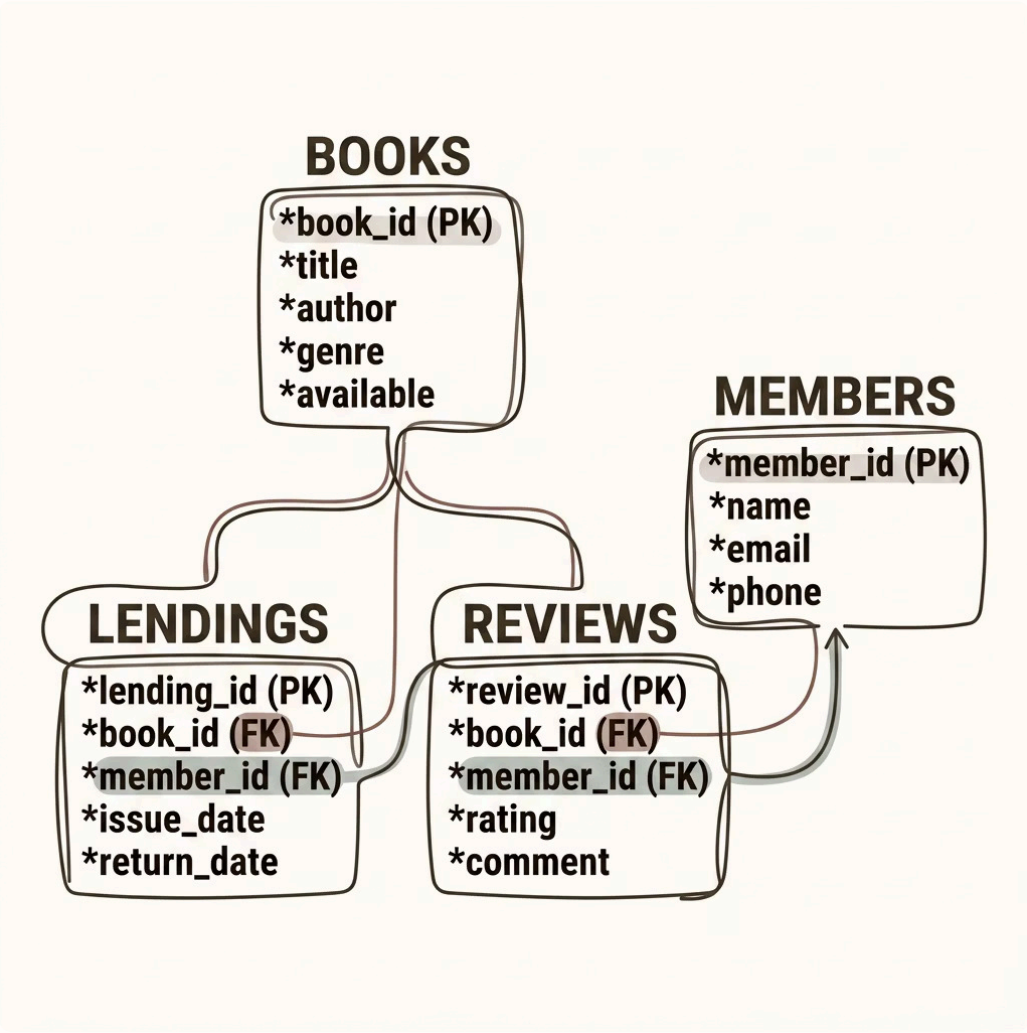
Application Layer

FastAPI handles routing, validation, and business logic via Pydantic schemas.

Data Layer

PostgreSQL stores all persistent data; SQLAlchemy manages ORM mappings.

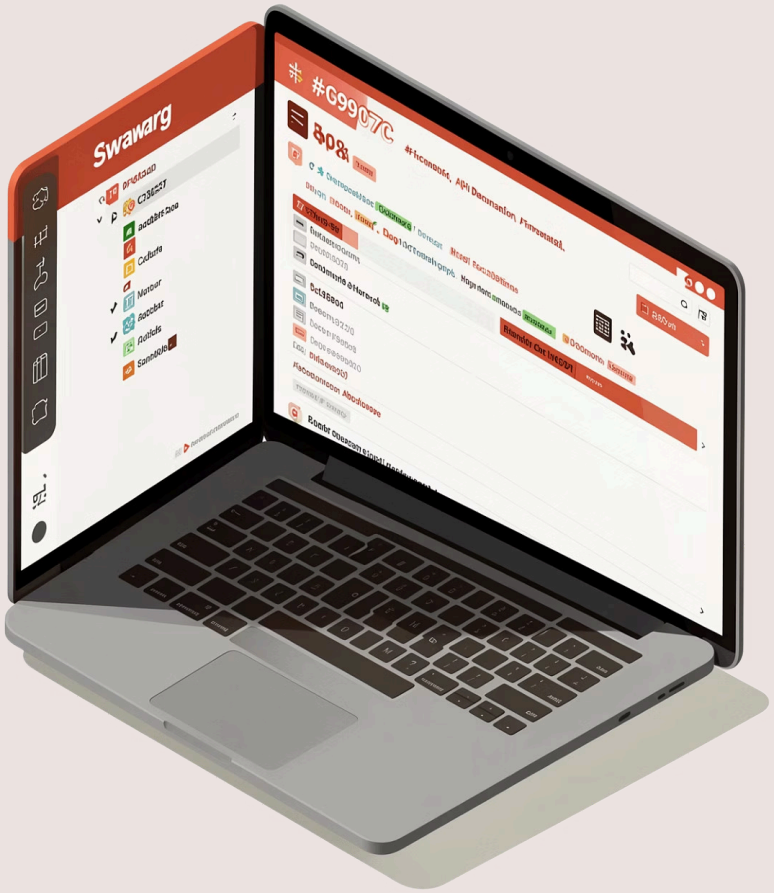
Database Design



Core Relationships

- **Books ↔️ Lendings**: One book can have many lending records over time.
- **Members ↔️ Lendings**: One member can borrow multiple books.
- **Books & Members ↔️ Reviews**: Each review links a member to a specific book with a rating.

> Foreign key constraints enforce referential integrity across all four tables.



API Demo

BookNest exposes a fully documented REST API via **Swagger UI** at `/docs` — auto-generated by FastAPI.

GET /books

Retrieve the full book catalog with availability status.

POST /lending

Issue a book to a registered member and log the date.

PUT /lending/{id}/return

Mark a book as returned and update inventory automatically.

POST /reviews

Submit a star rating and written review for a book.

Frontend Demo

Dashboard

Overview of total books, active members, and pending returns at a glance.

Book Catalog

Searchable, filterable list of all titles with availability indicators.

Lending System

Simple form to issue or return books, with member lookup and date auto-fill.

Design Philosophy

The frontend is intentionally minimal — prioritizing usability over visual complexity. Every interaction maps directly to a REST API call, keeping the UI stateless and the logic server-side.

- No framework dependency (pure HTML/CSS/JS)
- Mobile-responsive layout
- Instant feedback via API responses

Conclusion & Future Scope

What We Built

BookNest demonstrates a complete, working DBMS solution — from schema design and ORM mapping to REST API development and a functional web interface.

Future Roadmap

🔐 Authentication

JWT-based login for role-specific access (admin vs. member).

🎨 Enhanced UI

Migrate to a modern React or Vue frontend with richer UX.

🤖 Recommendations

ML-driven book suggestions based on borrowing history and ratings.

BookNest is architected for extensibility — each future feature plugs cleanly into the existing API layer.

01

Relational schema

Four normalized tables with enforced referential integrity.

02

REST API

Documented endpoints covering all CRUD operations.

03

Working frontend

Browser-based UI connecting directly to the live API.